

- `int (*func(int i))[10]` 表示数组中的元素是 `int` 类型。

使用尾置返回类型

C++
11

在 C++11 新标准中还有一种可以简化上述 `func` 声明的方法，就是使用尾置返回类型 (trailing return type)。任何函数的定义都能使用尾置返回，但是这种形式对于返回类型比较复杂的函数最有效，比如返回类型是数组的指针或者数组的引用。尾置返回类型跟在形参列表后面并以一个 `->` 符号开头。为了表示函数真正的返回类型跟在形参列表之后，我们在本应该出现返回类型的地方放置一个 `auto`：

230

```
// func 接受一个 int 类型的实参，返回一个指针，该指针指向含有 10 个整数的数组
auto func(int i) -> int(*)[10];
```

因为我们把函数的返回类型放在了形参列表之后，所以可以清楚地看到 `func` 函数返回的是一个指针，并且该指针指向了含有 10 个整数的数组。

使用 decltype

还有一种情况，如果我们知道函数返回的指针将指向哪个数组，就可以使用 `decltype` 关键字声明返回类型。例如，下面的函数返回一个指针，该指针根据参数 `i` 的不同指向两个已知数组中的某一个：

```
int odd[] = {1,3,5,7,9};
int even[] = {0,2,4,6,8};
// 返回一个指针，该指针指向含有 5 个整数的数组
decltype(odd) *arrPtr(int i)
{
    return (i % 2) ? &odd : &even; // 返回一个指向数组的指针
}
```

C++
11

`arrPtr` 使用关键字 `decltype` 表示它的返回类型是个指针，并且该指针所指的对象与 `odd` 的类型一致。因为 `odd` 是数组，所以 `arrPtr` 返回一个指向含有 5 个整数的数组的指针。有一个地方需要注意：`decltype` 并不负责把数组类型转换成对应的指针，所以 `decltype` 的结果是个数组，要想表示 `arrPtr` 返回指针还必须在函数声明时加一个 `*` 符号。

6.3.3 节练习

练习 6.36: 编写一个函数的声明，使其返回数组的引用并且该数组包含 10 个 `string` 对象。不要使用尾置返回类型、`decltype` 或者类型别名。

练习 6.37: 为上一题的函数再写三个声明，一个使用类型别名，另一个使用尾置返回类型，最后一个使用 `decltype` 关键字。你觉得哪种形式最好？为什么？

练习 6.38: 修改 `arrPtr` 函数，使其返回数组的引用。



6.4 函数重载

如果同一作用域内的几个函数名字相同但形参列表不同，我们称之为重载 (overloaded) 函数。例如，在 6.2.4 节 (第 193 页) 中我们定义了几个名为 `print` 的函数：

```
void print(const char *cp);
```

```
void print(const int *beg, const int *end);
void print(const int ia[], size_t size);
```

这些函数接受的形参类型不一样，但是执行的操作非常类似。当调用这些函数时，编译器会根据传递的实参类型推断想要的是哪个函数。 ◀ 231

```
int j[2] = {0,1};
print("Hello World");           // 调用 print(const char*)
print(j, end(j) - begin(j));    // 调用 print(const int*, size_t)
print(begin(j), end(j));        // 调用 print(const int*, const int*)
```

函数的名字仅仅是让编译器知道它调用的是哪个函数，而函数重载可以在一定程度上减轻程序员起名字、记名字的负担。



main 函数不能重载。

定义重载函数

有一种典型的数据库应用，需要创建几个不同的函数分别根据名字、电话、账户号码等信息查找记录。函数重载使得我们可以定义一组函数，它们的名字都是 lookup，但是查找的依据不同。我们能通过以下形式中的任意一种调用 lookup 函数：

```
Record lookup(const Account&);           // 根据 Account 查找记录
Record lookup(const Phone&);             // 根据 Phone 查找记录
Record lookup(const Name&);              // 根据 Name 查找记录
Account acct;
Phone phone;
Record r1 = lookup(acct);                // 调用接受 Account 的版本
Record r2 = lookup(phone);               // 调用接受 Phone 的版本
```

其中，虽然我们定义的三个函数各不相同，但它们都有同一个名字。编译器根据实参的类型确定应该调用哪一个函数。

对于重载的函数来说，它们应该在形参数量或形参类型上有所不同。在上面的代码中，虽然每个函数都只接受一个参数，但是参数的类型不同。

不允许两个函数除了返回类型外其他所有的要素都相同。假设有两个函数，它们的形参列表一样但是返回类型不同，则第二个函数的声明是错误的：

```
Record lookup(const Account&);
bool lookup(const Account&); // 错误：与上一个函数相比只有返回类型不同
```

判断两个形参的类型是否相异

有时候两个形参列表看起来不一样，但实际上是相同的：

```
// 每对声明的是同一个函数
Record lookup(const Account &acct);
Record lookup(const Account&); // 省略了形参的名字

typedef Phone Telno;
Record lookup(const Phone&);
Record lookup(const Telno&);    // Telno 和 Phone 的类型相同
```

在第一对声明中，第一个函数给它的形参起了名字，第二个函数没有。形参的名字仅仅起 ◀ 232

到帮助记忆的作用，有没有它并不影响形参列表的内容。

第二对声明看起来类型不同，但事实上 `Telno` 不是一种新类型，它只是 `Phone` 的别名而已。类型别名（参见 2.5.1 节，第 60 页）为已存在的类型提供另外一个名字，它并不是创建新类型。因此，第二对中两个形参的区别仅在于一个使用类型原来的名字，另一个使用它的别名，从本质上来说它们没什么不同。



重载和 `const` 形参

如 6.2.3 节（第 190 页）介绍的，顶层 `const`（参见 2.4.3 节，第 57 页）不影响传入函数的对象。一个拥有顶层 `const` 的形参无法和另一个没有顶层 `const` 的形参区分开来：

```
Record lookup(Phone);
Record lookup(const Phone);           // 重复声明了 Record lookup(Phone)

Record lookup(Phone*);
Record lookup(Phone* const);         // 重复声明了 Record lookup(Phone*)
```

在这两组函数声明中，每一组的第二个声明和第一个声明是等价的。

另一方面，如果形参是某种类型的指针或引用，则通过区分其指向的是常量对象还是非常量对象可以实现函数重载，此时的 `const` 是底层的：

```
// 对于接受引用或指针的函数来说，对象是常量还是非常量对应的形参不同
// 定义了 4 个独立的重载函数
Record lookup(Account&);              // 函数作用于 Account 的引用
Record lookup(const Account&);        // 新函数，作用于常量引用

Record lookup(Account*);              // 新函数，作用于指向 Account 的指针
Record lookup(const Account*);        // 新函数，作用于指向常量的指针
```

在上面的例子中，编译器可以通过实参是否是常量来推断应该调用哪个函数。因为 `const` 不能转换成其他类型（参见 4.11.2 节，第 144 页），所以我们只能把 `const` 对象（或指向 `const` 的指针）传递给 `const` 形参。相反的，因为非常量可以转换成 `const`，所以上面的 4 个函数都能作用于非常量对象或者指向非常量对象的指针。不过，如 6.6.1 节（第 220 页）将要介绍的，当我们传递一个非常量对象或者指向非常量对象的指针时，编译器会优先选用非常量版本的函数。

233

建议：何时不应该重载函数

尽管函数重载能在一定程度上减轻我们为函数起名字、记名字的负担，但是最好只重载那些确实非常相似的操作。有些情况下，给函数起不同的名字能使得程序更易理解。举个例子，下面是几个负责移动屏幕光标的函数：

```
Screen& moveHome();
Screen& moveAbs(int, int);
Screen& moveRel(int, int, string direction);
```

乍看上去，似乎可以把这组函数统一命名为 `move`，从而实现函数的重载：

```
Screen& move();
Screen& move(int, int);
Screen& move(int, int, string direction);
```

其实不然，重载之后这些函数失去了名字中本来拥有的信息。尽管这些函数确实都是在

移动光标，但是具体移动的方式却各不相同。以 `moveHome` 为例，它表示的是移动光标的一种特殊实例。一般来说，是否重载函数要看哪个更容易理解：

```
// 哪种形式更容易理解呢？
myScreen.moveHome(); // 我们认为应该是这一个！
myScreen.move();
```

`const_cast` 和重载

在 4.11.3 节（第 145 页）中我们说过，`const_cast` 在重载函数的情景中最有用。举个例子，回忆 6.3.2 节（第 201 页）的 `shorterString` 函数：

```
// 比较两个 string 对象的长度，返回较短的那个引用
const string &shorterString(const string &s1, const string &s2)
{
    return s1.size() <= s2.size() ? s1 : s2;
}
```

这个函数的参数和返回类型都是 `const string` 的引用。我们可以对两个非常量的 `string` 实参调用这个函数，但返回的结果仍然是 `const string` 的引用。因此我们需要一种新的 `shorterString` 函数，当它的实参不是常量时，得到的结果是一个普通的引用，使用 `const_cast` 可以做到这一点：

```
string &shorterString(string &s1, string &s2)
{
    auto &r = shorterString(const_cast<const string&>(s1),
                           const_cast<const string&>(s2));
    return const_cast<string&>(r);
}
```

在这个版本的函数中，首先将它的实参强制转换成对 `const` 的引用，然后调用了 `shorterString` 函数的 `const` 版本。`const` 版本返回对 `const string` 的引用，这个引用事实上绑定在了某个初始的非常量实参上。因此，我们可以再将其转换回一个普通的 `string&`，这显然是安全的。

调用重载的函数

定义了一组重载函数后，我们需要以合理的实参调用它们。**函数匹配**（function matching）是指一个过程，在这个过程中我们把函数调用与一组重载函数中的某一个关联起来，函数匹配也叫做**重载确定**（overload resolution）。编译器首先将调用的实参与重载集合中每一个函数的形参进行比较，然后根据比较的结果决定到底调用哪个函数。

234

在很多（可能是大多数）情况下，程序员很容易判断某次调用是否合法，以及当调用合法时应该调用哪个函数。通常，重载集中的函数区别明显，它们要不然是参数的数量不同，要不就是参数类型毫无关系。此时，确定调用哪个函数比较容易。但是在另外一些情况下要想选择函数就比较困难了，比如当两个重载函数参数数量相同且参数类型可以相互转换时（第 4.11 节，141 页）。我们将在 6.6 节（第 217 页）介绍当函数调用存在类型转换时编译器处理的方法。

现在我们需要掌握的是，当调用重载函数时有三种可能的结果：

- 编译器找到一个与实参**最佳匹配**（best match）的函数，并生成调用该函数的代码。
- 找不到任何一个函数与调用的实参匹配，此时编译器发出**无匹配**（no match）的错

误信息。

- 有多于一个函数可以匹配,但是每一个都不是明显的最佳选择。此时也将发生错误,称为二义性调用 (ambiguous call)。

6.4 节练习

练习 6.39: 说明在下面的每组声明中第二条声明语句是何含义。如果有非法的声明,请指出来。

- (a) `int calc(int, int);`
`int calc(const int, const int);`
- (b) `int get();`
`double get();`
- (c) `int *reset(int *);`
`double *reset(double *);`



6.4.1 重载与作用域



一般来说,将函数声明置于局部作用域内不是一个明智的选择。但是为了说明作用域和重载的相互关系,我们将暂时违反这一原则而使用局部函数声明。

对于刚接触 C++ 的程序员来说,不太容易理清作用域和重载的关系。其实,重载对作用域的一般性质并没有什么改变:如果我们在内层作用域中声明名字,它将隐藏外层作用域中声明的同名实体。在不同的作用域中无法重载函数名:

235

```
string read();
void print(const string &);
void print(double); // 重载 print 函数
void fooBar(int ival)
{
    bool read = false; // 新作用域:隐藏了外层的 read
    string s = read(); // 错误: read 是一个布尔值,而非函数
    // 不好的习惯:通常来说,在局部作用域中声明函数不是一个好的选择
    void print(int); // 新作用域:隐藏了之前的 print
    print("Value: "); // 错误: print(const string &)被隐藏掉了
    print(ival); // 正确:当前 print(int)可见
    print(3.14); // 正确:调用 print(int); print(double)被隐藏掉了
}
```

大多数读者都能理解调用 `read` 函数会引发错误。因为当编译器处理调用 `read` 的请求时,找到的是定义在局部作用域中的 `read`。这个名字是个布尔变量,而我们显然无法调用一个布尔值,因此该语句非法。

调用 `print` 函数的过程非常相似。在 `fooBar` 内声明的 `print(int)` 隐藏了之前两个 `print` 函数,因此只有一个 `print` 函数是可用的:该函数以 `int` 值作为参数。

当我们调用 `print` 函数时,编译器首先寻找对该函数名的声明,找到的是接受 `int` 值的那个局部声明。一旦在当前作用域中找到了所需的名称,编译器就会忽略掉外层作用域中的同名实体。剩下的工作就是检查函数调用是否有效了。

Note

在 C++ 语言中，名字查找发生在类型检查之前。

第一个调用传入一个字符串面值，但是当前作用域内 `print` 函数唯一的声明要求参数是 `int` 类型。字符串面值无法转换成 `int` 类型，所以这个调用是错误的。在外层作用域中的 `print(const string&)` 函数虽然与本次调用匹配，但是它已经被隐藏掉了，根本不会被考虑。

当我们为 `print` 函数传入一个 `double` 类型的值时，重复上述过程。编译器在当前作用域内发现了 `print(int)` 函数，`double` 类型的实参转换成 `int` 类型，因此调用是合法的。

假设我们把 `print(int)` 和其他 `print` 函数声明放在同一个作用域中，则它将成为另一种重载形式。此时，因为编译器能看到所有三个函数，上述调用的处理结果将完全不同：

```
void print(const string &);  
void print(double);           // print 函数的重载形式  
void print(int);              // print 函数的另一种重载形式  
void fooBar2(int ival)  
{  
    print("Value: ");          // 调用 print(const string &)  
    print(ival);               // 调用 print(int)  
    print(3.14);               // 调用 print(double)  
}
```

6.5 特殊用途语言特性

236

本节我们介绍三种函数相关的语言特性，这些特性对大多数程序都有用，它们分别是：默认实参、内联函数和 `constexpr` 函数，以及在程序调试过程中常用的一些功能。

6.5.1 默认实参

某些函数有这样一种形参，在函数的很多次调用中它们都被赋予一个相同的值，此时，我们把这个反复出现的值称为函数的**默认实参**（`default argument`）。调用含有默认实参的函数时，可以包含该实参，也可以省略该实参。

例如，我们使用 `string` 对象表示窗口的内容。一般情况下，我们希望该窗口的高、宽和背景字符都使用默认值。但是同时我们也应该允许用户为这几个参数自由指定与默认值不同的数值。为了使得窗口函数既能接纳默认值，也能接受用户指定的值，我们把它定义成如下的形式：

```
typedef string::size_type sz; // 关于 typedef 参见 2.5.1 节（第 60 页）  
string screen(sz ht = 24, sz wid = 80, char backgrnd = ' ');
```

其中我们为每一个形参都提供了默认实参，默认实参作为形参的初始值出现在形参列表中。我们可以为一个或多个形参定义默认值，不过需要注意的是，一旦某个形参被赋予了默认值，它后面的所有形参都必须有默认值。

使用默认实参调用函数

如果我们想使用默认实参，只要在调用函数的时候省略该实参就可以了。例如，